SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus	BL3 – Apply	5

	expressions to find members who borrowed books from all genres.		
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	**5**
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	**5**

Code of Conduct:

I DARA KUSUMANJALI (192425398L) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS

1. Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

Relations:

- Student(Student_ID, Name, Class_ID, Grade)
- Class(Class_ID, Class_Name, Teacher_ID)
- Teacher(Teacher_ID, Teacher_Name)

SQL:

```
SELECT s.Student_ID, s.Name, c.Class_Name,
    t.Teacher_Name, s.GradeFROM Student sJOIN Class c ON s.Class_ID = c.Class_IDJOIN Teacher t
ON c.Teacher_ID = t.Teacher_ID;
```

Explanation:

The query joins the three tables using their common IDs and displays the student name, class, class teacher, and grade.

Conclusion:

This query lists **all students with their class teacher and grade**.

2. A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

Hospital Appointment System

Relations:

- Patient(Patient_ID, Name)
- Doctor(Doctor_ID, Name)

- Appointment(App_ID, Patient_ID, Doctor_ID, App_Date)

SQL:

```
SELECT d.Name, COUNT(a.App_ID) AS AppointmentsFROM Doctor dJOIN Appointment a ON
d.Doctor_ID = a.Doctor_IDWHERE MONTH(a.App_Date) =
MONTH(CURRENT_DATE)GROUP BY d.Doctor_ID, d.NameHAVING COUNT(a.App_ID) >
10;
```

Conclusion:

Finds **doctors having more than 10 appointments this month**.

3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

E-Commerce – Products Never Ordered

Relations:

- Product(Product_ID, Product_Name)
- Orders(Order_ID, Product_ID)

SQL:

```
SELECT Product_ID, Product_NameFROM ProductWHERE Product_ID NOT IN
    (SELECT Product_ID FROM Orders);
```

Conclusion:

Finds **products that have never been ordered**.

4. For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

University Database

Relations:

- Student(SID, Name)
- Course(CID, CName)
- Enroll(SID, CID)

Relational Algebra:

DBMS ← πSID(σCName='DBMS'(Enroll ⋈ Course))

OS ← πSID(σCName='OS'(Enroll ⋈ Course))

Result ← DBMS ∩ OS

Conclusion:

Finds students enrolled in **both DBMS and OS**.

5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

Payroll System

View:

```
CREATE VIEW Low_Salary ASSELECT *FROM Employee eWHERE Salary < (
   SELECT AVG(Salary)
   FROM Employee
   WHERE Dept_ID = e.Dept_ID
);
```

Update Salary by 10%:

```
UPDATE EmployeeSET Salary = Salary * 1.10WHERE Emp_ID IN (SELECT Emp_ID FROM Low_Salary);
```

Conclusion:

Employees earning **below their department average** get a **10% salary increase**.

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

Banking System

Relations:

- Customer(Customer_ID, Name)
- Account(Account_ID, Customer_ID, Branch_ID, Balance)
- Transaction(Transaction_ID, Account_ID, Amount)
- Branch(Branch_ID, Branch_Name)

SQL:

```
SELECT c.Name, a.Account_ID, a.Balance,
    t.Amount, b.Branch_NameFROM Customer cJOIN Account a ON c.Customer_ID =
a.Customer_IDJOIN Transaction t ON a.Account_ID = t.Account_IDJOIN Branch b ON a.Branch_ID =
b.Branch_ID;
```

Conclusion:

Retrieves **customer, account, transaction, and branch details** using JOINs.

7. A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs.

Retail Store – Daily Sales

SQL:

```
SELECT Category, SUM(Amount) AS Total_SalesFROM SalesWHERE Sale_Date =
CURRENT_DATEGROUP BY CategoryHAVING SUM(Amount) > 10000;
```

Conclusion:

Displays categories with **daily sales above Rs. 10,000**.

8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

Library System

Relations:

- Member(Member_ID, Name)
- Book(Book_ID, Genre)
- Borrow(Member_ID, Book_ID)

Tuple Relational Calculus:

{ m | Member(m) AND
∀b (Book(b) →
∃br (Borrow(br) AND
br.Member_ID = m.Member_ID AND
br.Book_ID = b.Book_ID)) }

Conclusion:

Finds members who borrowed books from **all genres**.

9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

Inventory Management System

SQL:

```
CREATE TABLE Category (
    Category_ID INT PRIMARY KEY
);
CREATE TABLE Product (
    Product_ID INT PRIMARY KEY,
    Quantity INT CHECK (Quantity >= 0),
```

```
  Category_ID INT,
  FOREIGN KEY (Category_ID)
    REFERENCES Category(Category_ID)
);
```

CHECK enforcement:

INSERT INTO Product VALUES (1, -5, 1);

Rejected because quantity cannot be negative.

FOREIGN KEY enforcement:

INSERT INTO Product VALUES (2, 10, 99);

Rejected if Category_ID 99 does not exist.

Conclusion:

CHECK validates values, while FOREIGN KEY ensures valid relationships.

10. Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

HR System

Relational Schema:

- Employee(Emp_ID, Name, Dept_ID)
- Department(Dept_ID, Dept_Name)
- Attendance(Emp_ID, Date, Status)
- Performance(Emp_ID, Month, Score)

Materialized View – Attendance:

CREATE MATERIALIZED VIEW Monthly_Attendance ASSELECT Emp_ID, MONTH(Date) AS Month,
  COUNT(*) AS Present_DaysFROM AttendanceWHERE Status = 'Present'GROUP BY Emp_ID, MONTH(Date);

Materialized View – Performance:

CREATE MATERIALIZED VIEW Monthly_Performance ASSELECT Emp_ID, Month, AVG(Score) AS Avg_ScoreFROM PerformanceGROUP BY Emp_ID, Month;

Conclusion:

Materialized views store **monthly attendance and performance summaries** for faster reporting.